DEPARTMENT OF INFORMATION AND COMMUNICATION

PO Box 12363 Jacobs 4026
Durban
Tel: 031 907 7111

Internet Programming (IP3)

DEVELOPMENT SOFTWARE 3 (DSOF300)

Name	Student Number
AN Khomo	22407383
N Masinga	22426770
KA Manzini	22416153
ZT Khanyile	22320153
J Macuacua	22329264

Table of Contents

Introduction

Every university department needs a website that does more than just sit there. Students, prospective applicants, and staff members should be able to find what they need quickly, ask questions without frustration, and feel that the department is accessible and responsive. Unfortunately, many academic websites still operate like digital brochures. They present information, but they do not truly interact with the people who use the websites.

The ICT Department at Mangosuthu University of Technology currently has a website that serves its basic purpose of sharing departmental information. However, after spending time navigating through it, our group noticed several areas where the experience could be significantly better. The website feels static. You can read about programs/ courses and staff members, but you cannot easily ask a question, book an appointment, search for something specific, or receive any kind of intelligent assistance. The navigation can be confusing, and the layout does not always adapt well to mobile phones, which is how many students now access websites.

This project is our response to those limitations. We set out to redesign the ICT Department website into something smarter, more interactive, and genuinely useful. Instead of a static site, we built a full-stack web application. This means the website now has a working backend powered by Node.js and Express, a connected database that stores real information, and interactive features that respond to user input. We also added a simple AI-powered FAQ assistant to help users get instant answers to common questions.

The result is not just a prettier website. It is a functional, data-driven system that reflects what we have learned in Internet Programming 2 about frontend development, backend architecture, database integration, middleware, and user-centered design. More importantly, it is a website that finally works the way students and staff need it to work.

Problem Statement

The current ICT Department website exists, and it contains important information, but using it can be frustrating. Our group analyzed the website carefully, and we identified several specific problems that negatively affect the user experience.

First, the website is largely static. You can read content, but you cannot interact with the department through the website. There is no way to submit an enquiry online, no search function, and no automated system to answer frequently asked questions. **Second, navigation is not intuitive.** Important pages are not easy to find. A new student looking for staff contact details may need to click through multiple menus before succeeding.

Third, the website does not display well on all devices. When viewed on a smartphone, text becomes too small, layouts break, and buttons become hard to tap. Many students access the web primarily on their phones, so this is a real problem.

Fourth, there is no smart or intelligent functionality. Users cannot ask questions and receive instant answers. In 2026, students expect immediate responses.**Fifth, the website has no backend database connection.** Everything is hardcoded. If an announcement needs updating, someone must manually edit HTML files**.**

These problems matter because the ICT Department website is often the first point of contact between the university and prospective students. A website that is difficult to use, unresponsive, and static creates a poor impression. It also fails to support current students who need quick access to departmental services. Our project directly addresses each of these weaknesses by transforming the static website into a smart, interactive, and database-driven web application.

Project Objectives

1. Redesign the website using UXD principles (Chunking, Progressive Disclosure) so it is easy to use on any device.
2. Build a robust backend using Node.js and Express to process data.
3. Connect a database (Firebase Firestore) to securely store and retrieve departmental news and events.
4. Add middleware to handle logging, input validation, and errors.
5. Create a smart AI chatbot using the Google Gemini API to answer common prospectus questions.
6. Ensure full system integration across the frontend, backend, and database

Technologies Used

Category	Technologies
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6), marked.js
Backend	Node.js, Express.js, CORS
Database	Firebase Firestore (NoSQL), Firebase Admin SDK

Category	Technologies
Middleware	Custom Request Logger, Input Validator
AI Feature	Google Gemini 2.5 Flash API
Optimization & Deployment	Netlify

UXD Analysis and Frontend Redesign

We applied User Experience Design principles to fix the identified issues:

- **Simplified Navigation:** Cleaned up the top navigation and implemented a responsive Hamburger menu for mobile users. The footer was condensed from a cluttered list into a sleek 4-column layout.
- **Progressive Disclosure:** Long lists of driving directions on the Contact page were placed into interactive accordions, hiding secondary information until requested.
- **Mobile-First Design:** The Programme cards were converted into a horizontal swipe carousel on mobile to eliminate endless vertical scrolling.
- **Real Interactivity:** Modals were added for the Programmes section, allowing users to view NQF levels, credits, and admission requirements in a clean pop-up without navigating away from the page.

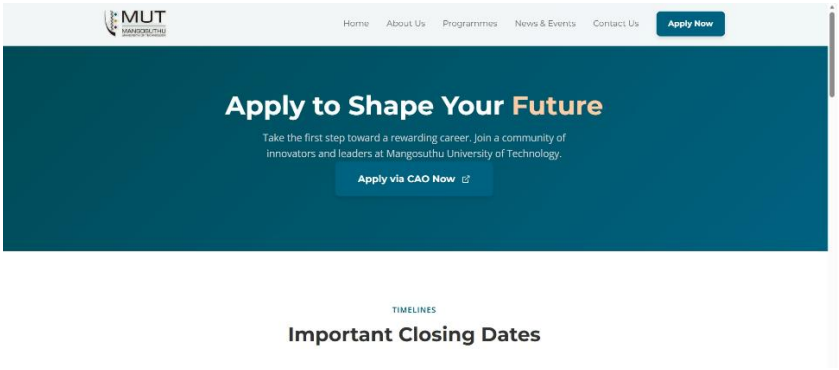


Figure1 Desktop Home Page

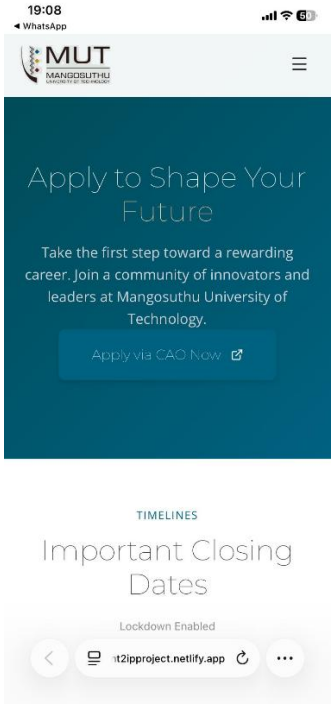
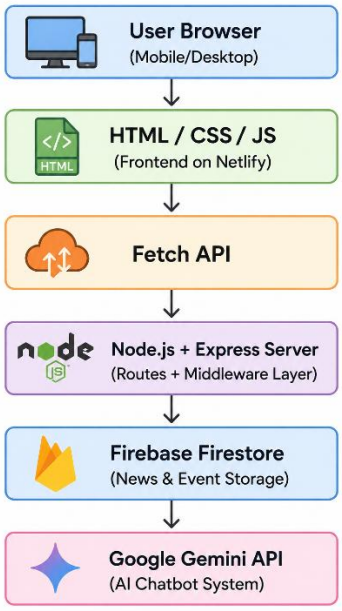


Figure2 Mobile Home Page

System Architecture

The following diagram demonstrates how the system components communicate with each other.



Backend Development (Node.js/Express)

We built the backend using Node.js and Express.js. This server handles data processing communicates with the Firebase database, and connects to the Gemini AI. The primary feature managed by the backend is the News & Events CRUD system.

Method	Endpoint	What It Does
GET	/api/news	Retrieves all active news articles from Firestore to display in the frontend grid.
POST	/api/news	Saves a new departmental news article (Create).
PUT	/api/news/:id	Updates details of an existing news article based on document ID.
DELETE	/api/news/:id	Removes an outdated article from the database.
POST	/api/chat	Sends a user query to the Gemini AI and returns the response.

Database Design and Integration

Unlike outdated SQL structures, modern web apps benefit from scalable NoSQL databases. We replaced the original concept of a MySQL database with **Firebase Firestore**.

- **Architecture:** Data is stored in a `news_events` collection. Each document contains fields for `title`, `date`, `description`, and `imageUrl`.
- **Admin Dashboard:** We built a hidden `admin.html` interface. When an administrator submits a new article, the frontend JavaScript sends a JSON payload to the Express server, which writes the document to Firestore in real-time.

Middleware Implementation

Middleware functions run between receiving a request and sending a response. We implemented crucial security and debugging layers:

1. **Global Request Logger:** Intercepts every incoming request, printing the HTTP method, route, and timestamp to the console to track traffic and debug routing issues.
2. **Input Validation (`validateNewsInput`):** Before the server saves an article to Firebase, this middleware checks `req.body`. If the title, date, or description are missing, it rejects the request with a `400 Bad Request` status, ensuring database integrity.

```
// Example: InputValidation Middleware
const validateNewsInput = (req, res, next) => {
    const { title, date, description } = req.body;
    if (!title || !date || !description) {
        return res.status(400).json({ error: "Missing required fields" });
    }
    next();
};
```

AI-Driven / Smart Feature (Gemini Integration)

We built the **MUT ICT Admissions Assistant**. Instead of basic keyword matching, this chatbot utilizes a Large Language Model (LLM).

- **How it Works:** The user types a question. JavaScript sends the query to `/api/chat`. The Express server connects to the **Google Gemini 2.5 Flash API** using a secure environment key.
- **Contextual Accuracy:** We used System Instructions to bind the AI to the MUT ICT prospectus, preventing hallucinations and ensuring it only answers questions related to admissions, CAO, and program requirements.

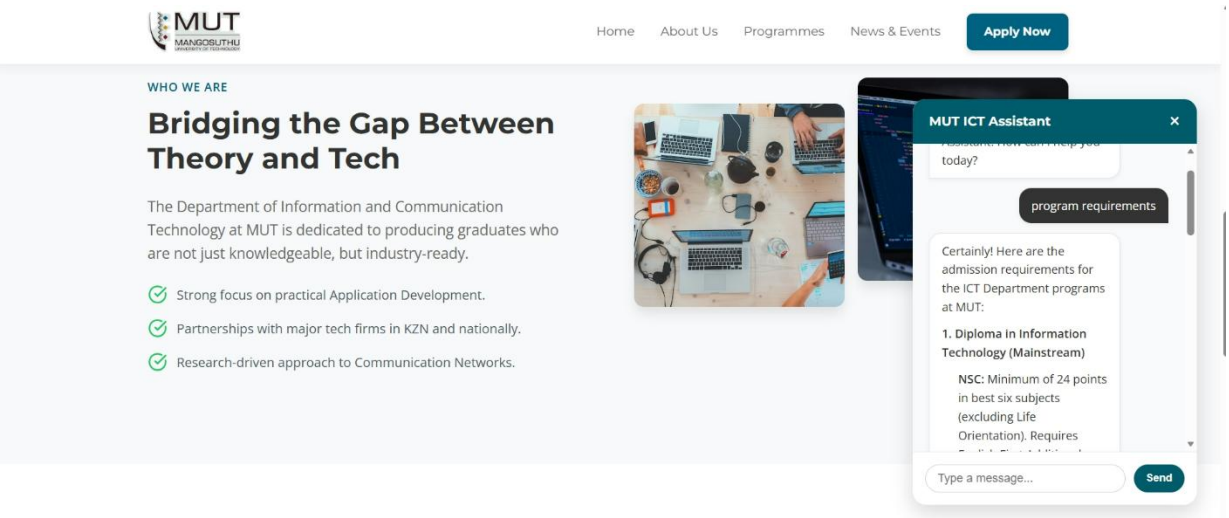


Figure3 Chat box demonstration.

Full System Integration & HTTP Flow

The system operates in a seamless loop:

1. **Frontend Request:** The browser uses the Fetch API to send a request (e.g., fetching news or sending a chat message).
2. **Backend Processing:** Express.js receives the request. Middleware logs it and validates the data.
3. **Database/AI Action:** The server reads/writes to Firebase or queries the Gemini API.
4. **Frontend Update:** The server returns a JSON response. The frontend JavaScript dynamically updates the DOM without reloading the page.

Group Member Contributions

Name	Student Number	Contribution
KA Manzini	22416153	Project Lead, Node.js/Express backend setup, Gemini AI API Integration.
N Masinga	22426770	Frontend HTML/CSS, UXD redesign, mobile responsiveness, Cloudinary optimization.
ZT Khanyile	22320153	Firebase Firestore database design, CRUD API routes.
AN Khomo	22407383	Middleware implementation (Logging, Validation), Error handling.
J Macuacua	22329264	Admin Dashboard frontend logic, JavaScript DOM manipulation (Modals/Tabs).

Challenges and Solutions

Challenge 1: CORS Blocking Frontend-Backend Communication

The problem: The frontend hosted on Netlify could not talk to our backend running locally or on a different port. Browsers blocked the requests for security reasons.

The solution: We installed the cors package and configured it properly on our Express server. We also had to update the allowed origins to include our Netlify URL. After a few failed attempts and reading the error messages carefully, we got it working.

Challenge 2: Firebase Environment Variables on Deployment

The problem: The Firebase Admin SDK requires a service account key file. We could not commit this file to GitHub because it contains sensitive credentials.

The solution: We stored the entire service account JSON as a string in an environment variable (.env file) and told Git to ignore it using .gitignore. On the deployed server, we set the same environment variable manually. The server parses the JSON string and initializes Firebase without needing the physical file.

Conclusion

Looking back at where we started, our group is genuinely proud of what we built. The ICT Department website at Mangosuthu University of Technology has been transformed from a static information page into a smart, interactive, and fully functional web application. We successfully implemented a responsive frontend with UXD principles (simplified navigation, mobile carousel, accordions, and modals), a robust Node.js/Express backend, Firebase Firestore for database-driven news management with full CRUD operations, and middleware for logging and input validation. The highlight of our project is the Gemini-powered AI chatbot, which provides instant, accurate answers to prospective student questions based on the MUT ICT prospectus. Every requirement of the assessment has been met, and all system components work together seamlessly.

This project taught us that building a full-stack application is not just about writing code—it is about making informed decisions and debugging systematically when things break. The system runs reliably, the user experience is genuinely better than the original website, and we can explain exactly how every piece works. If we had more time, we would add user authentication and a student enquiry system, but for now, we have succeeded in our goal: a website that finally works the way students and staff need it to work.

References

1. Express.js, 2026. *Express routing, middleware, and API development*. Available at: https://expressjs.com [Accessed 20 April 2026].

2. Firebase, 2026. *Cloud Firestore: NoSQL database for web applications*. Available at: https://firebase.google.com/docs/firestore [Accessed 21 April 2026].

3. Google AI Studio, 2026. *Gemini API documentation and system instructions*. Available at: https://ai.google.dev/gemini-api [Accessed 01 May 2026].

4. MDN Web Docs, 2026. *Fetch API: Making HTTP requests in JavaScript*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API [Accessed 20 April 2026].

5. Lecturer Notes, 2026. Internet Programming 2: Lesson 1-14. Mangosuthu University of Technology.